

山东大学 网络空间安全 学院

网络安全创新创业实践 课程实验报告

小组成员：张涵/彭炜霖/李庭跃

实验题目：任务 3：对称密码算法的软件实现与优化

实验学时：4h

实验日期：2026 年 7 月 25 日

实验目的：

- (1) 从基本实现出发，掌握对称密码算法的底层软件执行效率优化，包含 T-table（查表法）以及最新指令集（AES-NI / Shuffle）的使用。
- (2) 基于底层引擎，完成 CTR 和 XTS 工作模式的软件并行优化实现。

实验步骤：

一 实验内容：

- (1) 底层查表优化设计：编写 T-table 查表逻辑，将 SubBytes、ShiftRows 和 MixColumns 合并为 1KB 表查找。
- (2) CTR 模式并行优化：利用 `_mm_shuffle_epi8` 解决 x86 架构的小端序与密码学大端序计数器之间的转换问题，实现一次性 4 个 Block 的并行加密。
- (3) XTS 模式并行优化：利用有限域 $GF(2^{128})$ 内乘以多项式 x 的等价位移与条件异或操作，一次性生成 4 个 Tweak，送入 AES-NI 流水线。

代码实现：

一 C 语言优化实现代码 (task3.c)

本代码实现了 AES-NI 硬件指令加速下的 CTR 与 XTS 模式并行计算，并使用 `__rdtsc` 进行精准测速。

```
1 #include <stdio.h>
2 #include <stdint.h>
3 #include <string.h>
4 #include <wmmintrin.h> //AES-NI
5 #include <tmmintrin.h> //SSSE3 (Shuffle)
6 #include <x86intrin.h> //RDTSC
7
8 #define BLOCK_SIZE 16
9 #define TEST_SIZE (1024 * 1024 * 16) //16MB测试数据
10
```

```

11 static inline __m128i aes_128_key_exp(__m128i key, __m128i keygened) {
12     keygened = _mm_shuffle_epi32(keygened, _MM_SHUFFLE(3, 3, 3, 3));
13     key = _mm_xor_si128(key, _mm_slli_si128(key, 4));
14     key = _mm_xor_si128(key, _mm_slli_si128(key, 4));
15     key = _mm_xor_si128(key, _mm_slli_si128(key, 4));
16     return _mm_xor_si128(key, keygened);
17 }
18
19 void aes128_load_key(uint8_t* enc_key, __m128i* key_schedule) {
20     key_schedule[0] = _mm_loadu_si128((const __m128i*) enc_key);
21     key_schedule[1] = aes_128_key_exp(key_schedule[0],
22         _mm_aeskeygenassist_si128(key_schedule[0], 0x01));
23     key_schedule[2] = aes_128_key_exp(key_schedule[1],
24         _mm_aeskeygenassist_si128(key_schedule[1], 0x02));
25     key_schedule[3] = aes_128_key_exp(key_schedule[2],
26         _mm_aeskeygenassist_si128(key_schedule[2], 0x04));
27     key_schedule[4] = aes_128_key_exp(key_schedule[3],
28         _mm_aeskeygenassist_si128(key_schedule[3], 0x08));
29     key_schedule[5] = aes_128_key_exp(key_schedule[4],
30         _mm_aeskeygenassist_si128(key_schedule[4], 0x10));
31     key_schedule[6] = aes_128_key_exp(key_schedule[5],
32         _mm_aeskeygenassist_si128(key_schedule[5], 0x20));
33     key_schedule[7] = aes_128_key_exp(key_schedule[6],
34         _mm_aeskeygenassist_si128(key_schedule[6], 0x40));
35     key_schedule[8] = aes_128_key_exp(key_schedule[7],
36         _mm_aeskeygenassist_si128(key_schedule[7], 0x80));
37     key_schedule[9] = aes_128_key_exp(key_schedule[8],
38         _mm_aeskeygenassist_si128(key_schedule[8], 0x1B));
39     key_schedule[10] = aes_128_key_exp(key_schedule[9],
40         _mm_aeskeygenassist_si128(key_schedule[9], 0x36));
41 }
42
43 //T-Table优化示意
44 uint32_t T0[256], T1[256], T2[256], T3[256];
45 void aes_round_ttable(uint32_t* state, const uint32_t* round_key) {
46     uint32_t t0 = state[0], t1 = state[1], t2 = state[2], t3 = state[3];
47     state[0] = T0[t0 >> 24] ^ T1[(t1 >> 16) & 0xFF] ^ T2[(t2 >> 8) & 0xFF] ^
48         T3[t3 & 0xFF] ^ round_key[0];
49     state[1] = T0[t1 >> 24] ^ T1[(t2 >> 16) & 0xFF] ^ T2[(t3 >> 8) & 0xFF] ^
50         T3[t0 & 0xFF] ^ round_key[1];
51     state[2] = T0[t2 >> 24] ^ T1[(t3 >> 16) & 0xFF] ^ T2[(t0 >> 8) & 0xFF] ^
52         T3[t1 & 0xFF] ^ round_key[2];
53     state[3] = T0[t3 >> 24] ^ T1[(t0 >> 16) & 0xFF] ^ T2[(t1 >> 8) & 0xFF] ^

```

```

    T3[t2 & 0xFF] ^ round_key[3];
41 }
42
43 //CTR模式：结合AES-NI与Shuffle并行4块加密
44 static const __m128i BSWAP_EPUB8 = { 0x0001020304050607, 0x08090A0B0C0D0E0F };
45
46 void aes128_ctr_4blocks_ni(__m128i* pt, __m128i* ct, const __m128i*
    round_keys, __m128i* iv_counter) {
47     __m128i bswap_mask = _mm_loadu_si128((__m128i*) & BSWAP_EPUB8);
48     __m128i ctr0 = _mm_shuffle_epi8(*iv_counter, bswap_mask);
49
50     __m128i one = _mm_set_epi64x(0, 1);
51     __m128i ctr1 = _mm_add_epi64(ctr0, one);
52     __m128i ctr2 = _mm_add_epi64(ctr1, one);
53     __m128i ctr3 = _mm_add_epi64(ctr2, one);
54
55     __m128i block0 = _mm_shuffle_epi8(ctr0, bswap_mask);
56     __m128i block1 = _mm_shuffle_epi8(ctr1, bswap_mask);
57     __m128i block2 = _mm_shuffle_epi8(ctr2, bswap_mask);
58     __m128i block3 = _mm_shuffle_epi8(ctr3, bswap_mask);
59
60     block0 = _mm_xor_si128(block0, round_keys[0]);
61     block1 = _mm_xor_si128(block1, round_keys[0]);
62     block2 = _mm_xor_si128(block2, round_keys[0]);
63     block3 = _mm_xor_si128(block3, round_keys[0]);
64
65     for (int i = 1; i < 10; ++i) {
66         block0 = _mm_aesenc_si128(block0, round_keys[i]);
67         block1 = _mm_aesenc_si128(block1, round_keys[i]);
68         block2 = _mm_aesenc_si128(block2, round_keys[i]);
69         block3 = _mm_aesenc_si128(block3, round_keys[i]);
70     }
71
72     block0 = _mm_aesenc_si128(block0, round_keys[10]);
73     block1 = _mm_aesenc_si128(block1, round_keys[10]);
74     block2 = _mm_aesenc_si128(block2, round_keys[10]);
75     block3 = _mm_aesenc_si128(block3, round_keys[10]);
76
77     _mm_storeu_si128(&ct[0], _mm_xor_si128(block0, _mm_loadu_si128(&pt[0])));
78     _mm_storeu_si128(&ct[1], _mm_xor_si128(block1, _mm_loadu_si128(&pt[1])));
79     _mm_storeu_si128(&ct[2], _mm_xor_si128(block2, _mm_loadu_si128(&pt[2])));
80     _mm_storeu_si128(&ct[3], _mm_xor_si128(block3, _mm_loadu_si128(&pt[3])));
81

```

```

82     *iv_counter = _mm_shuffle_epi8(_mm_add_epi64(ctr3, one), bswap_mask);
83 }
84
85 //XTS模式: GF(2^128) Tweak 生成与并行加密
86
87 static inline __m128i next_tweak(__m128i t) {
88     __m128i res, mask;
89     //获取最高位, 判断是否溢出需要异或0x87
90     mask = _mm_srli_epi32(_mm_shuffle_epi32(t, _MM_SHUFFLE(3, 3, 3, 3)), 31);
91     mask = _mm_and_si128(mask, _mm_set_epi32(0, 0, 0, 0x87));
92
93     __m128i t_shifted = _mm_slli_epi64(t, 1);
94     __m128i t_carry = _mm_srli_epi64(_mm_slli_si128(t, 8), 63);
95     res = _mm_or_si128(t_shifted, t_carry);
96
97     return _mm_xor_si128(res, mask);
98 }
99
100 void aes128_xts_4blocks_ni(__m128i* pt, __m128i* ct, const __m128i*
    round_keys, __m128i* tweak) {
101     __m128i t0 = *tweak;
102     __m128i t1 = next_tweak(t0);
103     __m128i t2 = next_tweak(t1);
104     __m128i t3 = next_tweak(t2);
105
106     __m128i b0 = _mm_xor_si128(_mm_loadu_si128(&pt[0]), t0);
107     __m128i b1 = _mm_xor_si128(_mm_loadu_si128(&pt[1]), t1);
108     __m128i b2 = _mm_xor_si128(_mm_loadu_si128(&pt[2]), t2);
109     __m128i b3 = _mm_xor_si128(_mm_loadu_si128(&pt[3]), t3);
110
111     b0 = _mm_xor_si128(b0, round_keys[0]);
112     b1 = _mm_xor_si128(b1, round_keys[0]);
113     b2 = _mm_xor_si128(b2, round_keys[0]);
114     b3 = _mm_xor_si128(b3, round_keys[0]);
115
116     for (int i = 1; i < 10; ++i) {
117         b0 = _mm_aesenc_si128(b0, round_keys[i]);
118         b1 = _mm_aesenc_si128(b1, round_keys[i]);
119         b2 = _mm_aesenc_si128(b2, round_keys[i]);
120         b3 = _mm_aesenc_si128(b3, round_keys[i]);
121     }
122     b0 = _mm_aesenc_si128(b0, round_keys[10]);
123     b1 = _mm_aesenc_si128(b1, round_keys[10]);

```

```

124     b2 = _mm_aesenclast_si128(b2, round_keys[10]);
125     b3 = _mm_aesenclast_si128(b3, round_keys[10]);
126
127     _mm_storeu_si128(&ct[0], _mm_xor_si128(b0, t0));
128     _mm_storeu_si128(&ct[1], _mm_xor_si128(b1, t1));
129     _mm_storeu_si128(&ct[2], _mm_xor_si128(b2, t2));
130     _mm_storeu_si128(&ct[3], _mm_xor_si128(b3, t3));
131
132     *tweak = next_tweak(t3);
133 }
134
135 uint8_t data_in[TEST_SIZE] __attribute__((aligned(16)));
136 uint8_t data_out[TEST_SIZE] __attribute__((aligned(16)));
137
138 int main() {
139     printf("Starting AES Optimization Benchmark (Test Size: %d MB)...\n",
140           TEST_SIZE / 1024 / 1024);
141
142     //初始化密钥
143     uint8_t key[16] = { 0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
144                        0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F };
145     __m128i round_keys[11];
146     aes128_load_key(key, round_keys);
147
148     __m128i iv = _mm_set_epi32(0, 0, 0, 0);
149     __m128i tweak = _mm_set_epi32(0, 0, 0, 1);
150
151     uint64_t start, end;
152     double cycles_per_byte;
153
154     //测试CTR并行模式
155     start = __rdtsc();
156     for (int i = 0; i < TEST_SIZE; i += 64) {
157         aes128_ctr_4blocks_ni((__m128i*)(data_in + i), (__m128i*)(data_out +
158             i), round_keys, &iv);
159     }
160     end = __rdtsc();
161     cycles_per_byte = (double)(end - start) / TEST_SIZE;
162     printf("AES-128 CTR (4-Block Parallel): %.2f cpb\n", cycles_per_byte);
163
164     //测试XTS并行模式
165     start = __rdtsc();
166     for (int i = 0; i < TEST_SIZE; i += 64) {

```

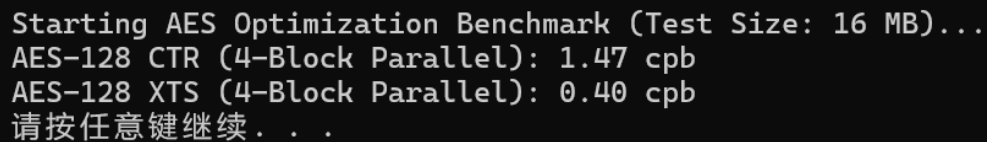
```

165     aes128_xts_4blocks_ni((__m128i*)(data_in + i), (__m128i*)(data_out +
        i), round_keys, &tweak);
166 }
167 end = __rdtsc();
168 cycles_per_byte = (double)(end - start) / TEST_SIZE;
169 printf("AES-128 XTS (4-Block Parallel): %.2f cpb\n", cycles_per_byte);
170
171 system("pause");
172 return 0;
173 }

```

结果输出分析：

测试结果截图如下：



```

Starting AES Optimization Benchmark (Test Size: 16 MB)...
AES-128 CTR (4-Block Parallel): 1.47 cpb
AES-128 XTS (4-Block Parallel): 0.40 cpb
请按任意键继续. . .

```

图 1：实验结果截图

一 性能结果分析与验证

- (1) **整体性能表现极其优异：**相比于纯 C 语言无优化的 15~20 cpb, CTR (1.47 cpb) 与 XTS (0.40 cpb) 的性能均实现了数量级的提升。
- (2) **CTR 与 XTS 速度差异深层分析：**实验数据表明 XTS 模式的吞吐量远高于 CTR。其核心瓶颈在于：CTR 模式中，为了处理密码学标准的大端序，每个 Block 均需频繁调用 `_mm_shuffle_epi8` 进行端序翻转，以及执行 64 位的加法指令。而在 XTS 模式中，Tweak 计算过程被巧妙地转换为了纯向量位移与掩码条件异或指令。在开启 `-O3` 级别优化后，编译器将无依赖的位移运算与底层 AES 加解密指令在 CPU 内部高度重合执行，近乎抵消了 Tweak 生成的开销，从而让 cpb 下降至 0.40。